

MongoDB Tutorial: Inserting Documents into Collections

1. Overview

This tutorial explains how to insert documents into a MongoDB collection. It covers:

- How MongoDB stores data inside databases and collections
 - How to insert single and multiple documents
 - Syntax and practical usage of `insertOne()` and `insertMany()`
 - How MongoDB generates `_id` fields automatically
 - How to verify inserted data using `find()`
-

2. MongoDB Data Structure Recap

2.1 Database

A **database** stores collections and keeps related data grouped.

2.2 Collection

A **collection** contains multiple documents, similar to tables in relational databases.

2.3 Document

A **document** is a JSON-like object containing fields and values.

Example:

```
{
  "name": "Akshay Kumar",
  "age": 25,
  "class": "BCA"
}
```

MongoDB internally stores data in BSON (Binary JSON) but exposes it as JSON when shown in the shell.

3. Insert Operations in MongoDB

MongoDB provides two commands for inserting documents:

3.1 `insertOne()`

Used to insert **a single document**.

3.2 `insertMany()`

Used to insert **multiple documents at once** in an array format.

4. Syntax and Usage of `insertOne()`

4.1 General Syntax

```
db.<collectionName>.insertOne({  
  field1: value1,  
  field2: value2,  
  ...  
})
```

Notes:

- Fields are written in JSON format.
 - String values must be written inside quotes.
 - Numeric values are written without quotes.
-

4.2 Practical Example

Step 1: Start Mongo Shell

```
mongosh
```

Step 2: Create/Select Database

```
use school
```

Step 3: Create Collection

```
db.createCollection("students")
```

Step 4: Insert One Document

```
db.students.insertOne({
  name: "Akshay Kumar",
  age: 25,
  class: "BCA"
})
```

Result Example

```
{
  "acknowledged": true,
  "insertedId": ObjectId("...")
}
```

Step 5: View Inserted Data

```
db.students.find()
```

5. Syntax and Usage of `insertMany()`

5.1 General Syntax

```
db.<collectionName>.insertMany([
  { fieldA: valueA, fieldB: valueB },
  { fieldA: valueA2, fieldB: valueB2 },
  ...
])
```

Notes:

- Uses **square brackets []** for an array of documents.
- Each document is a JSON object separated by commas.
- You can insert any number of documents (even thousands) at once.

5.2 Practical Example

```
db.students.insertMany([
  {
    name: "Salman Khan",
    age: 28,
    class: "BBA"
  }
])
```

```
    },  
    {  
      name: "John Abraham",  
      age: 30,  
      class: "BCom"  
    }  
  ]  
})
```

Insert Result

```
{  
  "acknowledged": true,  
  "insertedIds": {  
    "0": ObjectId("..."),  
    "1": ObjectId("...")  
  }  
}
```

View All Records

```
db.students.find()
```

6. Document Structure and Auto-generated `_id`

Every time you insert a document, MongoDB automatically adds:

```
"_id": ObjectId("624c...")
```

The `_id` serves as a unique identifier.

You do not need to create it manually.

7. Example: Incorrect Data Entry

MongoDB does not restrict field types unless schema validation is configured.

For example:

```
db.students.insertOne({  
  name: "Test User",  
})
```

```
    age: 22,  
    class: 200 // Incorrect type, but MongoDB will still store it  
  })
```

Result:

```
db.students.find()
```

Shows:

```
{  
  "name": "Test User",  
  "age": 22,  
  "class": 200  
}
```

To prevent such mistakes, MongoDB schema validation can be used (covered in next lessons).

8. Summary of Key Concepts

Command	Purpose
<code>insertOne()</code>	Insert a single document into a collection
<code>insertMany()</code>	Insert multiple documents into a collection
<code>find()</code>	Retrieve documents from the collection
<code>_id</code>	Auto-generated unique identifier for each document

9. Complete Code Flow (All Commands Together)

```
mongosh  
use school  
db.createCollection("students")  
  
db.students.insertOne({  
  name: "Akshay Kumar",  
  age: 25,  
  class: "BCA"  
})  
  
db.students.insertOne({
```

```
    name: "Sunil Shetty",
    age: 23,
    class: "BTech"
  })

  db.students.insertMany([
    {
      name: "Salman Khan",
      age: 28,
      class: "BBA"
    },
    {
      name: "John Abraham",
      age: 30,
      class: "BCom"
    }
  ])

  db.students.find()
```
